Spécification — Client Wall

Version : 1.0
Statut : Draft
Date : 2026-03-27

Présentation du produit

Client Wall est une web app permettant aux **freelances, entreprises et associations** de créer des espaces dédiés pour gérer leurs clients et leurs collaborateurs.

Sur ces espaces, clients et collaborateurs peuvent :

Fonctionnalité	Description
📄 **Contrat signé**	Consulter et signer le contrat lié à la mission
🧾 **Factures**	Accéder aux factures émises
💳 **Paiement en ligne**	Régler une facture directement via **Stripe**
📊 **Avancement du projet**	Suivre l'état d'avancement en temps réel
📁 **Upload de fichiers**	Déposer des documents dans l'espace partagé
💬 **Messagerie**	Discuter directement avec le freelance ou l'équipe

> Client Wall positionne le **projet** comme unité centrale de collaboration — et non le client — ce qui permet de gérer des missions impliquant plusieurs interlocuteurs au sein d'un même espace structuré.

Table des matières

1. Vue d'ensemble du modèle de données
2. Workspace
3. Projet
4. Zone
5. Utilisateurs
6. Rôles et permissions
7. Documents
8. Messagerie

9. Facturation
10. Points d'attention
11. Stack technique
12. Équipe et capacités
13. Planning

---

1. Vue d'ensemble du modèle de données

Le système repose sur une **arborescence à trois niveaux** :

```
Workspace
└── Projet
    └── Zone
```

Niveau	Portée	Rôle
Workspace	Organisation	Structure l'entité et ses membres
Projet	Activité métier	Regroupe les ressources d'une mission
Zone	Visibilité et accès	Contrôle qui voit quoi

---

2. Workspace

2.1 Définition

Le Workspace représente une **organisation** (entreprise, freelance, association). C'est l'entité racine du système.

2.2 Administration

Un Workspace est géré par :

- **1 propriétaire** — unique, responsable de l'entité
- **0 à n gestionnaires** — droits d'administration étendus

> **Règle par défaut :** les gestionnaires du Workspace sont automatiquement gestionnaires de tous les projets qu'il contient.

2.3 Responsabilités

- Contient **0 à n projets**

- Centralise la gestion des **collaborateurs internes**
- Définit les **règles globales** : accès, configuration, intégrations

---

3. Projet

3.1 Définition

Un Projet représente une **affaire métier spécifique** (mission, contrat, prestation). Il constitue une unité autonome et contient :

- Documents métier
- Factures
- États d'avancement
- Échanges (messagerie)

3.2 Gestionnaires de projet

Source	Gestionnaires
Automatique	Propriétaire du Workspace + gestionnaires du Workspace
Manuel	Tout utilisateur ajouté explicitement (même s'il n'est pas gestionnaire Workspace)

> **Contrainte forte :** il est impossible de déléguer complètement un projet. Le Workspace conserve toujours un droit de contrôle.

---

4. Zone

4.1 Définition

Une Zone est un **sous-espace d'un projet** qui détermine la visibilité des ressources et les droits d'accès pour un groupe d'utilisateurs donné.

4.2 Zones par défaut

Tout projet contient par défaut :

- **1 zone client**
- **1 zone collaborateur**

4.3 Zones multiples

Il est possible de créer plusieurs zones par projet :

- Plusieurs zones client
- Plusieurs zones collaborateur

Chaque zone représente ainsi un **contexte de partage spécifique**.

---

5. Utilisateurs

5.1 Types

Type	Description
Client	Utilisateur externe, rattaché à une zone client
Collaborateur	Utilisateur interne, rattaché à une zone collaborateur

5.2 Règles d'appartenance

- Une **zone client** contient 0 à n clients
- Une **zone collaborateur** contient 0 à n collaborateurs
- **Un client ne peut jamais appartenir à une zone collaborateur** (et inversement)

5.3 Représentation d'un client

Dans Client Wall, un **client** désigne une **entité** (personne morale ou physique) liée au projet — et non un compte utilisateur individuel.

Cette entité peut être représentée par **plusieurs comptes utilisateurs** : des interlocuteurs côté client (chef de projet, comptable, directeur…) qui ont chacun accès à l'espace partagé.

> La notion de "plusieurs clients dans une zone" reflète donc plusieurs **représentants d'un même client**, et non plusieurs clients distincts sur un même projet.

Lorsque plusieurs comptes coexistent dans une zone client :

- Ils **partagent la même visibilité** sur les ressources exposées
- Ils peuvent avoir des **rôles différents** (ex : un compte peut payer une facture, un autre peut seulement consulter)

---

6. Rôles et permissions

6.1 Modèle de rôle

Un rôle est un ensemble de couples **(permission, ressource)**.

6.2 Portée des rôles

- Les rôles sont **globaux au projet** — pas de rôle spécifique à une zone
- Une zone peut néanmoins être **associée à un rôle**

6.3 Attribution

Un utilisateur peut obtenir un rôle via deux canaux :

- **Héritage par zone** — le rôle est défini au niveau de la zone
- **Attribution directe** — le rôle est assigné directement à l'utilisateur

6.4 Évaluation des permissions

Le système repose sur trois types de valeur de permission, évalués selon l'ordre de priorité suivant :

```
block  >  allow  >  deny
```

Type	Description	Priorité
block	Interdit explicitement une action — l'emporte sur tout `allow`	Absolue
allow	Autorise une action	Haute
deny	Refuse une action (état implicite par défaut)	Basse

Règles d'évaluation :

- Les permissions issues de tous les rôles d'un utilisateur sont **combinées par union**
- Si un `block` est présent sur une action, il **écrase tout** `allow`, quelle que soit la source
- En l'absence de `block`, un `allow` l'emporte sur un `deny`
- `deny` est l'état implicite — l'absence de permission équivaut à un refus

> **Cas d'usage typique du `block` :** restreindre l'accès à une ressource sensible (ex : marges, notes internes) pour un utilisateur qui hériterait sinon d'un `allow` via son rôle Workspace.

6.5 Permissions disponibles

Actions :

- Lecture
- Écriture
- Ajout

- Modification
- Suppression
- Accès messagerie (lecture / écriture)

Ressources concernées :

- Documents
- Messages
- Éléments métier (factures, avancement, feuilles de temps…)

---

7. Documents

7.1 Rattachement

Tout document est **rattaché à un projet**.

7.2 Catégories

Catégorie	Exemples
Téléversés par l'organisation	Fichiers, contrats, annexes
Générés par le système	Factures, états d'avancement, feuilles de temps
Gérés par l'outil	Signature de contrat
Téléversés par le client	Pièces justificatives, livrables

7.3 Documents générés par le système

Ces documents sont produits automatiquement par Client Wall à partir des données du projet. Ils constituent le cœur de la valeur ajoutée de la plateforme côté client.

7.3.1 Facture

Rôle : document financier transmis au client pour déclencher un paiement.

Données portées :

Champ	Description
Numéro de facture	Identifiant unique, généré automatiquement
Date d'émission	Date de création de la facture
Date d'échéance	Date limite de paiement
Émetteur	Informations de l'organisation (nom, adresse, SIRET…)
Destinataire	Informations du client facturé
Lignes de facturation	Intitulé, quantité, prix unitaire, total HT
TVA	Taux et montant applicable
Total TTC	Montant final dû
Statut	Brouillon / Émise / Payée / En retard
Lien de paiement	URL Stripe générée à l'émission

Cycle de vie :

```
Brouillon → Émise → Payée
                  ↘ En retard (si échéance dépassée)
```

Règles métier :

- Une facture est rattachée au **projet** et adressée au **client en tant qu'entité**
- Elle peut être exposée dans la **zone client** — tous les représentants du client la voient, mais seuls ceux avec la permission de paiement peuvent la régler via Stripe
- La responsabilité du paiement est gérée en interne par le client, via les rôles attribués dans la zone
- Une fois émise, elle ne peut plus être modifiée (générer un avoir si nécessaire)
- Le passage au statut **Payée** est déclenché automatiquement via webhook Stripe

Visibilité :

- Gestionnaires du projet : lecture complète + émission
- Zone client exposée : lecture + paiement en ligne
- Marges et remises internes : restreintes aux gestionnaires

7.3.2 État d'avancement

Rôle : document de suivi synthétique communiqué au client pour l'informer de la progression de la mission.

Données portées :

Champ	Description
Titre / Période	Intitulé de la livraison ou période couverte
Date de publication	Date à laquelle l'état est partagé
Phases / Jalons	Liste des étapes avec statut (À faire / En cours / Terminé)
Pourcentage global	Avancement global de la mission (0–100 %)
Description narrative	Texte libre de synthèse rédigé par le gestionnaire
Prochaines étapes	Ce qui est prévu pour la période suivante
Pièces jointes	Livrables intermédiaires ou captures associées

Cycle de vie :

```
Brouillon → Publié → Archivé
```

Règles métier :

- Créé manuellement par un gestionnaire, sur la base de l'avancement réel du projet
- Peut être publié dans une **zone client** pour partage en lecture seule
- Les versions précédentes sont conservées et consultables (historique)
- Un état d'avancement publié **ne peut pas être modifié** — une nouvelle version doit être créée

Visibilité :

- Gestionnaires : création, publication, historique complet
- Zone client exposée : lecture de la version publiée uniquement

---

7.3.3 Feuille de temps

Rôle : document de suivi interne recensant le temps passé par les collaborateurs sur le projet. Peut servir de base à la facturation au temps passé.

Données portées :

Champ	Description
Collaborateur	Nom du collaborateur concerné
Période	Semaine, mois ou période personnalisée
Lignes de temps	Date, tâche, durée (en heures), commentaire
Total heures	Cumul sur la période
Taux horaire	Optionnel — utilisé pour le calcul de coût
Coût total	Calculé si taux horaire renseigné (interne)
Statut	Saisie en cours / Soumise / Validée

Cycle de vie :

```
Saisie en cours → Soumise → Validée
```

Règles métier :

- Saisie par les collaborateurs, validée par un gestionnaire
- Peut être utilisée pour **pré-remplir une facture** (import des lignes de temps)
- Les coûts internes (taux horaire, marge) sont **restreints aux gestionnaires** — jamais exposés en zone client
- Une feuille de temps validée **ne peut plus être modifiée** par le collaborateur

Visibilité :

- Collaborateurs : saisie et lecture de leurs propres lignes
- Gestionnaires : lecture complète, validation, accès aux coûts
- Zone client : **non exposée par défaut** — exposition possible en lecture partielle (heures uniquement, sans coût)

7.4 Visibilité des documents

- Par défaut, un document est **visible uniquement dans son projet**
- Il peut être **exposé dans une ou plusieurs zones**, sans duplication (référence unique)
- Certains documents peuvent être **restreints aux gestionnaires** du projet (ex : notes internes, marges, coûts des feuilles de temps)

8. Messagerie

8.1 Messagerie par zone

Chaque zone possède une messagerie propre, organisée en **threads**. C'est le **mode de consultation par défaut** pour les utilisateurs : ils accèdent aux échanges dans le contexte de leur zone.

8.2 Log global de messagerie

Un **log global** de tous les messages échangés au niveau du projet existe et est accessible aux gestionnaires du projet.

Caractéristique	Description
Portée	Tous les threads de toutes les zones du projet
Accès	Gestionnaires du projet uniquement
Mode de consultation	Vue consolidée, en lecture — pas l'interface principale
Usage typique	Audit, supervision, recherche transversale

> Le log global **n'est pas le point d'entrée naturel** de la messagerie. Les utilisateurs consultent leurs échanges via la messagerie de leur zone. Le log global est un outil de supervision réservé aux gestionnaires.

8.3 Messagerie personnelle

Chaque utilisateur dispose d'une **messagerie individuelle**, indépendante des zones.

8.4 Règles d'accès

Situation	Comportement par défaut
Échange client ↔ collaborateur	**Interdit**
Échange client ↔ collaborateur	Possible avec une **autorisation explicite**

8.5 Conformité RGPD

- Un message **ne peut pas être supprimé pour tous les participants**
- La suppression est **locale** (pour l'expéditeur) ou **contrôlée** selon les règles définies

9. Facturation

- Les factures sont **rattachées au projet** et adressées au **client dans son ensemble** (l'entité), pas à un compte utilisateur spécifique
- Le modèle est **centré projet** : une facture concerne la relation contractuelle entre l'organisation et son client

Responsabilité du paiement :

La responsabilité du règlement d'une facture relève de l'**organisation interne au client**. Client Wall ne gère pas cette répartition : c'est au client de désigner, via les rôles de sa zone, quel(s) compte(s) ont la permission de déclencher un paiement via Stripe.

> **Exemple :** un chef de projet côté client peut consulter les factures, mais seul le compte "comptabilité" dispose de la permission de paiement.

Conséquences sur la conception :

- La permission de **paiement en ligne** (Stripe) doit être un droit explicitement attribuable à un rôle dans la zone client
- L'interface doit afficher clairement le statut de la facture à tous les membres de la zone, mais restreindre l'action de paiement aux comptes autorisés
- Aucune logique de répartition ou de quote-part entre représentants n'est gérée par Client Wall

---

10. Points d'attention

10.1 Modèle de permissions (block > allow > deny)

Le modèle à trois niveaux est expressif et couvre les cas d'usage courants :

- `allow` pour ouvrir un accès
- `deny` comme état par défaut implicite
- `block` pour une restriction explicite et absolue, qui ne peut pas être contournée par un `allow` hérité

> **Point de vigilance :** les `block` doivent être posés de manière intentionnelle. Un `block` mal placé peut verrouiller un accès attendu sans message d'erreur explicite pour l'utilisateur — l'UI doit rendre visible pourquoi une action est bloquée.

10.2 Plusieurs représentants dans une zone client

Un client (l'entité) peut avoir plusieurs comptes dans sa zone. Ces représentants partagent la même visibilité, mais peuvent avoir des rôles différents. Cela implique :

- Définir clairement quels rôles donnent accès au **paiement** (permission Stripe)
- La répartition interne des responsabilités côté client est **hors du périmètre de Client Wall**
- L'UI doit rendre lisible pour chaque compte ce qu'il peut ou ne peut pas faire

10.3 Facturation adressée à l'entité client

Les factures sont adressées au client en tant qu'entité globale. Client Wall ne gère pas de logique de quote-part ou de répartition entre représentants. L'organisation interne du paiement est une responsabilité côté client, à piloter via les permissions de rôle dans la zone.

10.4 Multi-zones

Les zones multiples offrent une granularité fine, mais impliquent :

- Une **complexité accrue** de la gestion des accès
- Une **interface de configuration** claire et guidée

10.5 Log global de messagerie

Le log global existe mais n'est pas le mode de consultation principal. Il est réservé aux gestionnaires pour des usages de supervision et d'audit. Cette distinction doit être claire dans l'architecture de navigation :

- **Zone → thread** : chemin naturel pour tous les utilisateurs
- **Log global projet** : outil avancé, accessible depuis une vue dédiée aux gestionnaires

---

11. Stack technique

11.1 Vue d'ensemble

Couche	Technologie
Backend	Spring Boot (Java)
Base de données	PostgreSQL
Frontend	React + Vite
Paiement	Stripe (webhooks)

11.2 Backend — Spring Boot

- API REST exposée au frontend
- Gestion des entités du domaine : Workspace, Projet, Zone, Utilisateur, Rôle, Permission
- Moteur d'évaluation des permissions (`block > allow > deny`)
- Intégration Stripe via webhook pour les mises à jour de statut de facturation
- Génération des documents système (factures PDF, états d'avancement, feuilles de temps)

11.3 Base de données — PostgreSQL

- Modèle relationnel centré projet
- Gestion des droits au niveau applicatif (pas de RLS PostgreSQL)
- Stockage des métadonnées de documents (les fichiers binaires sont externalisés : S3 ou équivalent)
- Historisation des états (factures, feuilles de temps validées, versions d'avancement)

11.4 Frontend — React + Vite

- SPA avec routing côté client
- Deux interfaces distinctes selon le profil :
  - **Interface organisation** : gestion Workspace, Projets, Zones, Documents, Messagerie, Facturation
  - **Interface client** : espace dédié en lecture/interaction (contrat, factures, avancement, upload, messagerie)
- Authentification via token (JWT)

---

12. Équipe et capacités

12.1 Composition

Personne	Profil	Compétences
P1	Fullstack	Frontend · Backend · Design
P2	Fullstack	Frontend · Backend · Design
P3	Backend / DevOps	Backend · DevOps

12.2 Capacités par domaine

Domaine	Personnes disponibles
SGBD	P1, P2
Design	P1, P2
Frontend	P1, P2
Backend	P1, P2, P3
DevOps / Déploiement	P2, P3

---

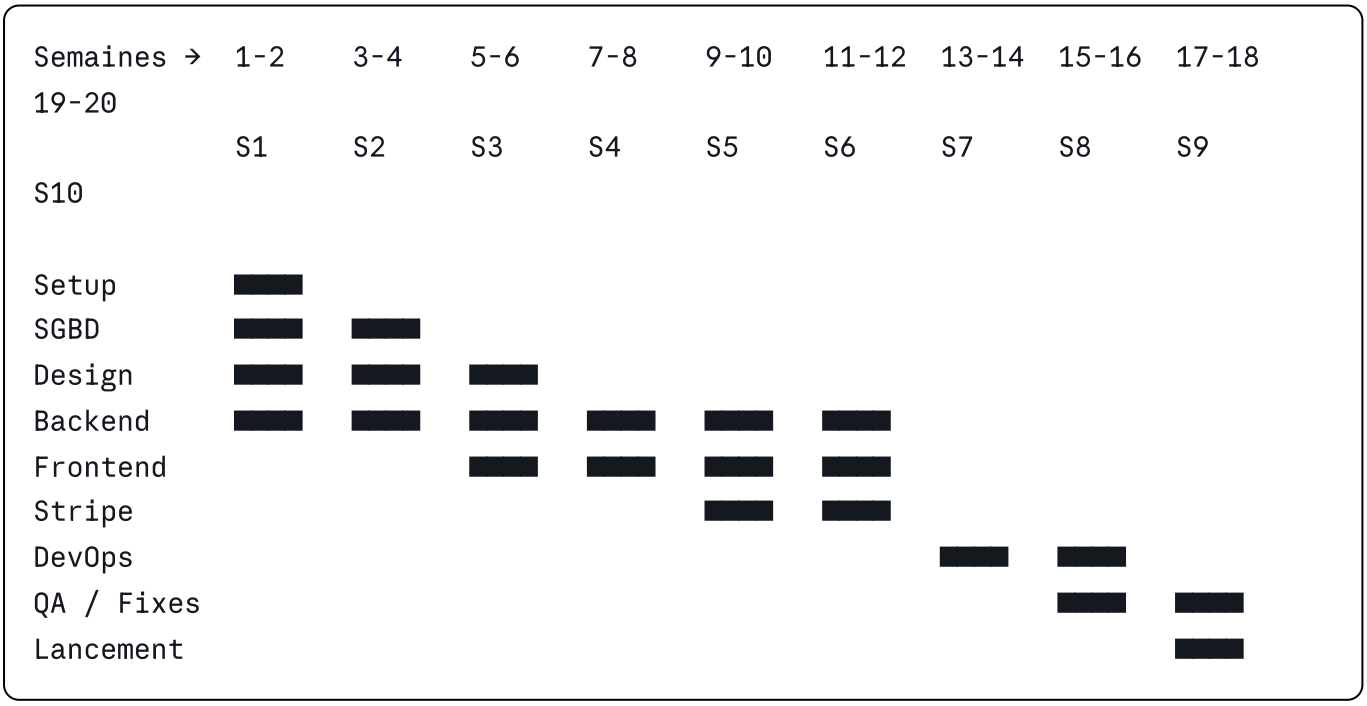
13. Planning

13.1 Hypothèses

- Sprints de **2 semaines**
- **10 sprints** → durée totale estimée : **20 semaines (~5 mois)**
- Pas de période de découverte produit incluse (spec réalisée en amont)
- Les phases Design et Backend peuvent être parallélisées dès le début

---

13.2 Vue par phase



13.3 Détail par sprint

S1 — Setup & fondations *(sem. 1-2)*

Domaine	Tâches	Qui
SGBD	Modélisation initiale du schéma (Workspace, Projet, Zone, User, Rôle)	P1, P2
Backend	Initialisation Spring Boot, structure des modules, auth JWT	P2, P3
Design	Audit concurrentiel, moodboard, premières wireframes (navigation globale)	P1, P2
DevOps	Setup repo, CI de base (build + lint), environnement de dev	P3

S2 — Domaine métier core *(sem. 3-4)*

Domaine	Tâches	Qui
SGBD	Finalisation du schéma, migrations Flyway, seeds de dev	P1, P2
Backend	CRUD Workspace, Projet, Zone — moteur de permissions (`block > allow > deny`)	P1, P2, P3
Design	Wireframes détaillés : interface organisation (dashboard, projet, zone)	P1, P2

S3 — Utilisateurs, rôles & design system *(sem. 5-6)*

Domaine	Tâches	Qui
Backend	Gestion utilisateurs (invitation, rôles, appartenance aux zones)	P2, P3
Design	Design system (composants, tokens, typographie) + maquettes interface client	P1, P2
Frontend	Initialisation React + Vite, routing, intégration design system, écran de connexion	P1, P2

S4 — Documents & frontend core *(sem. 7-8)*

Domaine	Tâches	Qui
Backend	Module documents (upload, métadonnées, visibilité par zone, stockage S3)	P1, P3
Frontend	Interface organisation : Workspace, Projets, Zones, gestion membres	P1, P2
Backend	Génération PDF facture (structure, données, template)	P2, P3

S5 — Messagerie *(sem. 9-10)*

Domaine	Tâches	Qui
Backend	Messagerie par zone (threads), messagerie personnelle, log global projet	P1, P2, P3
Frontend	Interface documents (liste, upload, prévisualisation, exposition par zone)	P1, P2
Backend	Règles d'accès messagerie (autorisation explicite client ↔ collaborateur)	P3

S6 — Stripe & facturation *(sem. 11-12)*

Domaine	Tâches	Qui
Backend	Intégration Stripe (création lien de paiement, webhook statut Payée)	P2, P3
Frontend	Interface messagerie (threads par zone, messagerie personnelle)	P1, P2
Backend	Module feuille de temps (saisie, validation, import vers facture)	P1, P3

S7 — État d'avancement & espace client *(sem. 13-14)*

Domaine	Tâches	Qui
Backend	Module état d'avancement (création, publication, versioning, historique)	P1, P3
Frontend	Interface facturation (liste, statuts, lien Stripe, permissions paiement)	P1, P2
Frontend	Espace client : factures, avancement, contrat, upload, messagerie	P2, P3

S8 — DevOps & déploiement *(sem. 15-16)*

Domaine	Tâches	Qui
DevOps	Infrastructure production (VPS / cloud, Docker, PostgreSQL managé)	P2, P3
DevOps	Pipeline CI/CD complet (build, test, deploy automatisé)	P3
DevOps	Monitoring (logs, alertes, uptime), backup PostgreSQL automatisé	P2, P3
Frontend	Feuille de temps UI (saisie collaborateur, validation gestionnaire)	P1, P2

S9 — QA & stabilisation *(sem. 17-18)*

Domaine	Tâches	Qui
QA	Tests end-to-end scénarios critiques (permissions, Stripe, documents)	P1, P2, P3
Backend	Corrections, optimisations requêtes PostgreSQL, sécurité API	P2, P3
Frontend	Corrections UI, responsive, accessibilité de base	P1, P2
DevOps	Tests de charge, validation environnement prod	P3

S10 — Lancement *(sem. 19-20)*

Domaine	Tâches	Qui
Prod	Mise en production, bascule DNS, monitoring actif	P2, P3
Documentation	Guide utilisateur, documentation API interne	P1, P2
QA	Smoke tests post-déploiement, corrections critiques	P1, P2, P3

13.4 Risques planning

Risque	Impact	Mitigation
Complexité du moteur de permissions	Retard S2-S3	Prototyper et valider le modèle dès S1 avec des cas de test

Risque	Impact	Mitigation
Intégration Stripe (webhooks, edge cases)	Retard S6	Utiliser Stripe CLI en dev dès S1, prévoir un sprint de marge
Génération PDF (mise en page, données)	Retard S4	Utiliser une lib éprouvée (JasperReports, iText ou Flying Saucer)
Scope creep sur l'espace client	Retard S7	Définir un MVP strict de l'espace client avant S7
Environnement prod instable	Retard lancement	Déployer un environnement staging dès S5

Synthèse architecturale

Client Wall repose sur une architecture :

Dimension	Caractéristique
Multi-tenant	Via le Workspace (une organisation = un workspace)
Multi-scope	Via le Projet (une mission = un projet)
Multi-visibility	Via les Zones (clients et collaborateurs cloisonnés)
Permissions	Basées sur les rôles, évaluation par union
Paiement	Intégré via Stripe, déclenché depuis l'espace client

C'est un socle solide pour un outil de collaboration structuré. Sa robustesse dépend de la maîtrise de trois aspects critiques :

1. La **logique de permissions** — bien anticiper les combinaisons possibles
2. La **visibilité des données** — définir explicitement ce qui est exposé et à qui
3. Les **cas multi-clients** — prévoir une logique métier claire pour la facturation et les droits

---

Fin de spécification — v1.0